

ProductivityGuard

Earn your leisure time. Study first, scroll later.

A Windows desktop productivity system that tracks your activity, converts study time into "Coin" currency, and uses those coins to unlock distracting apps and websites. No coins = no Instagram. Earn them by studying.

The Idea

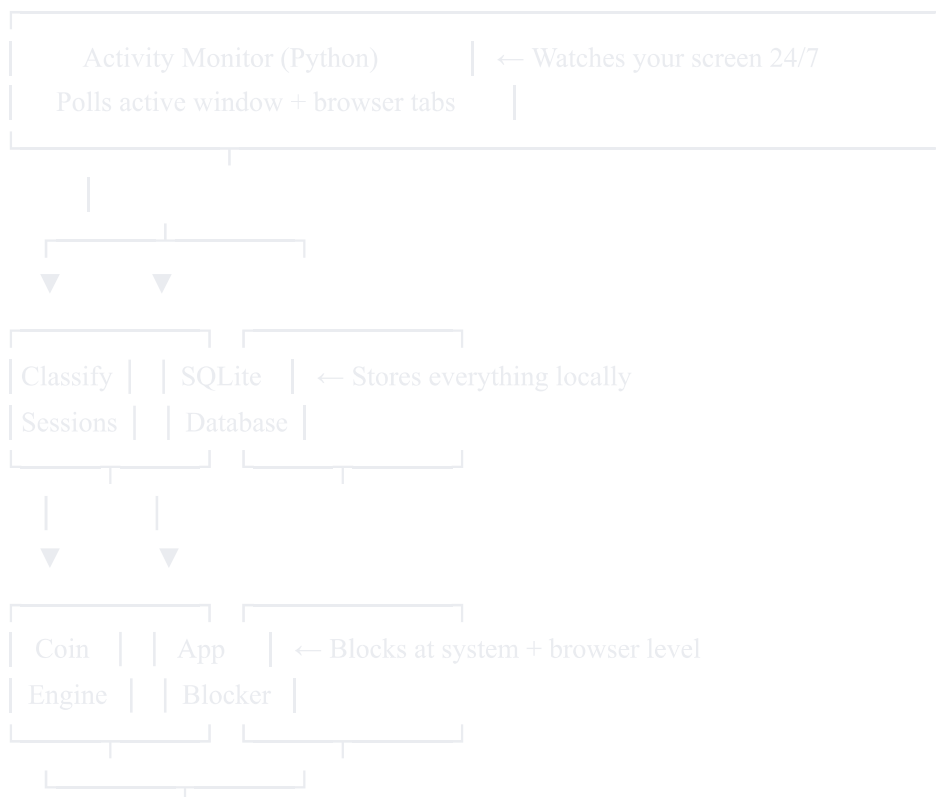
Most productivity apps work by blocking things. You feel restricted. You resent it. You uninstall.

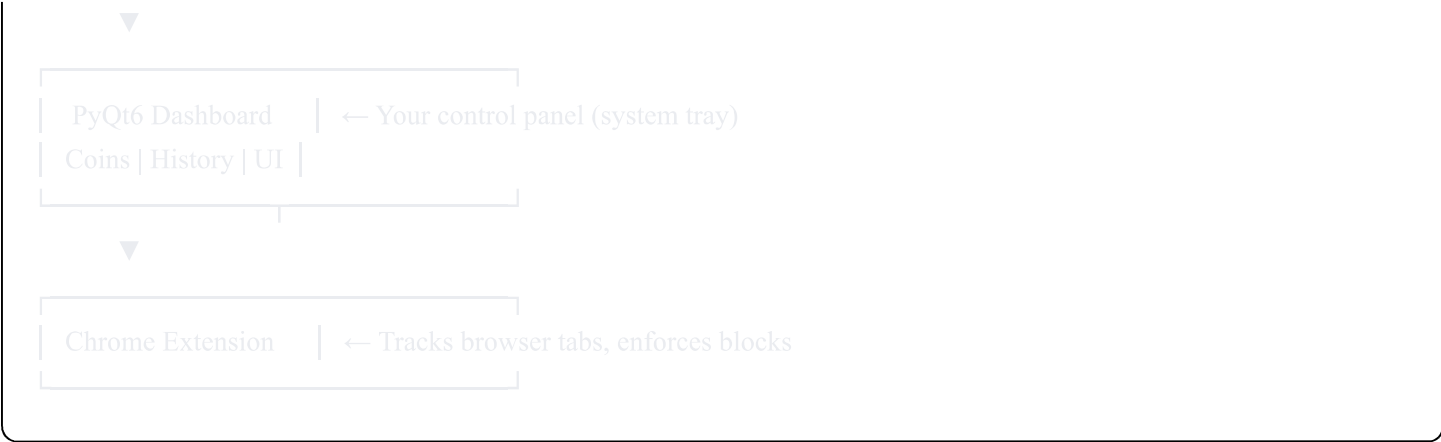
ProductivityGuard flips the model:

-  **Study for 3 hours** → earn 30 minutes of free time (coins)
-  **Spend coins** to unlock YouTube, Instagram, games — guilt-free
-  **Coins run out** → apps lock automatically
-  **Repeat daily** — discipline becomes a habit, not a punishment

You are never restricted from leisure. You simply have to **earn** it first.

Architecture





🔧 Tech Stack

Layer	Technology	Purpose	Cost
Activity Monitor	Python + <code>pywin32</code>	Read active Windows processes	Free
Database	SQLite	Store sessions, coins, history	Free built-in
Classifier	Python keyword rules	Label study vs timepass	Free
Coin Engine	Pure Python	Calculate and manage coins	Free
App Blocker	Python (hosts file)	System-level site blocking	Free
Dashboard UI	PyQt6	Desktop GUI app	Free
Local API	FastAPI	Bridge Python ↔ browser	Free
Browser Extension	JavaScript (MV3)	Tab tracking + blocking	Free
Packaging	PyInstaller	Build .exe for distribution	Free
IDE	Cursor (recommended)	AI-assisted development	Free tier

Everything is free and open source.

💰 Coin Economy

Earn Rate: 6 minutes of study = 1 minute of coins
Example: 3 hours study (180 min) = 30 minutes coins

Rules:

- Minimum study block: 15 minutes continuous (no gaming)
- Coins expire at midnight (no hoarding)
- Max daily cap: 120 minutes (2 hours)
- Minimum spend: 5 minutes per session

You configure all of these in `config/settings.json`.

Project Structure

ProductivityGuard/

```
|
|
|— core/
|   |— tracker.py      # Polls active window every 5 seconds
|   |— classifier.py   # Study vs timepass classification
|   |— coin_engine.py  # Earn / spend / balance logic
|   |— blocker.py      # Hosts file editing + app killing
|
|— database/
|   |— db.py           # SQLite connection + CRUD helpers
|   |— schema.sql      # All table definitions
|   |— productivity.db  # Auto-created on first run
|
|— api/
|   |— server.py       # FastAPI on localhost:5000
|
|— ui/
|   |— main_window.py  # PyQt6 dashboard window
|   |— tray.py         # System tray icon + menu
|   |— assets/         # Icons
|
|— extension/
|   |— manifest.json   # Chrome Extension Manifest V3
|   |— background.js   # Tab URL service worker
|   |— content.js      # Page detection
|   |— popup.html      # Extension popup UI
|
|— config/
|   |— settings.json    # Earn rate, caps, expiry rules
|   |— study_list.json  # Sites/apps counted as study
|   |— block_list.json  # Sites/apps to block
|
```

```
|— tests/
|   |— test_tracker.py
|   |— test_coin_engine.py
|   |— test_classifier.py
|
|— docs/
|   |— PIPELINE.md      # Full dev pipeline (this project)
|
|— main.py              # Entry point
|— requirements.txt     # Python dependencies
|— .gitignore
|— README.md           # This file
```

Build Pipeline — Start to End

Phase 0 — Setup (Day 1)

Goal: Development environment ready.

Tools needed:

- Python 3.11+ — python.org
- Cursor IDE — cursor.sh (free tier)
- Git + GitHub account
- Chrome browser (for extension testing)

Steps:

```
bash
```

```
# 1. Clone / create your repo
git init ProductivityGuard
cd ProductivityGuard

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Verify setup
python -c "import pywin32; print('Windows API: OK')"
python -c "import PyQt6; print('UI: OK')"
python -c "import fastapi; print('API: OK')"
```

Phase 1 — Activity Tracker (Week 1)

Goal: Python service that watches your screen and logs everything to SQLite.

Files to build:

- `database/schema.sql` — create all tables
- `database/db.py` — connect and query SQLite
- `core/tracker.py` — poll active window every 5s
- `core/classifier.py` — label each session

Key Python libraries:

```
pywin32    → win32gui.GetForegroundWindow() to get active app
psutil     → process name from window handle
pygetwindow → window title reading
```

Milestone test:

```
bash

python core/tracker.py

# Should print: "chrome.exe | YouTube - How to learn Python | timepass"
# every 5 seconds, and write rows to SQLite sessions table
```

How classifier works:

```
python
```

```
STUDY_DOMAINS = ["coursera.org", "khanacademy.org", "github.com",
                 "stackoverflow.com", "docs.python.org", "kaggle.com"]

STUDY_APPS = ["code.exe", "cursor.exe", "pycharm64.exe", "jupyter.exe"]

TIMEPASS_DOMAINS = ["instagram.com", "youtube.com", "reddit.com",
                   "twitter.com", "netflix.com"]

# YouTube exception: youtube.com/watch?v= in a specific browser = classify by watch time pattern
```

Phase 2 — Coin Engine (Week 2)

Goal: Calculate coins from study sessions. Store balance. Allow spending.

Files to build:

- `core/coin_engine.py` — all coin logic

Core logic:

```
python
```

```
def calculate_coins(study_seconds: int) -> int:
    """Every 6 minutes of study = 1 minute of coins."""
    if study_seconds < 900: # Less than 15 min = no coins (anti-gaming)
        return 0
    return study_seconds // 6

def spend_coins(site: str, seconds: int) -> bool:
    """Returns True if coins available and deducted. False if broke."""
    balance = get_balance()
    if balance >= seconds:
        deduct(seconds, source=site)
        return True
    return False
```

Milestone test:

```
bash
```

```
python -c "  
from core.coin_engine import calculate_coins  
print(calculate_coins(10800)) # 3hrs → should print 1800 (30 min)  
print(calculate_coins(600)) # 10min → should print 0 (below threshold)  
"
```

Phase 3 — App Blocker (Week 3, hardest phase)

Goal: Actually block sites and apps. Unblock when coins spent.

Files to build:

- `core/blocker.py` — hosts file modifier
- `api/server.py` — FastAPI bridge

Two-layer blocking:

Layer A — System level (hosts file):

```
python  
  
# Requires running as Administrator  
HOSTS_PATH = r"C:\Windows\System32\drivers\etc\hosts"  
  
def block_site(domain: str):  
    with open(HOSTS_PATH, 'a') as f:  
        f.write(f"\n127.0.0.1 {domain}")  
        f.write(f"\n127.0.0.1 www.{domain}")  
  
def unblock_site(domain: str):  
    # Remove the lines added above  
    ...
```

Layer B — Browser extension (JavaScript):

```
javascript
```

```
// background.js checks every tab URL against block list
chrome.tabs.onUpdated.addListener(async (tabId, changeInfo, tab) => {
  if (changeInfo.url) {
    const response = await fetch('http://localhost:5000/can_browse', {
      method: 'POST',
      body: JSON.stringify({ url: changeInfo.url })
    });
    const { allowed } = await response.json();
    if (!allowed) {
      chrome.tabs.update(tabId, { url: 'blocked.html' });
    }
  }
});
```

FastAPI bridge:

```
python

# api/server.py
from fastapi import FastAPI
app = FastAPI()

@app.post("/can_browse")
def can_browse(payload: dict):
    domain = extract_domain(payload["url"])
    if domain in block_list and get_balance() <= 0:
        return {"allowed": False, "balance": 0}
    return {"allowed": True, "balance": get_balance()}

@app.post("/spend")
def spend(payload: dict):
    success = spend_coins(payload["site"], payload["seconds"])
    return {"success": success, "balance": get_balance()}
```

Milestone test:

- Add instagram.com to block list with 0 coins
- Visit instagram.com in Chrome
- Should redirect to blocked page

Goal: A real desktop app you can see and interact with.

Files to build:

- `ui/main_window.py` — main PyQt6 window
- `ui/tray.py` — system tray icon

Key screens:

1. **Home** — Today's coin balance (big number), study time today, quick stats
2. **History** — Table of sessions (time, app, duration, category, coins earned)
3. **Spend Coins** — List of blocked apps, spend N minutes to unlock
4. **Settings** — Edit study list, block list, earn rate, expiry rules

PyQt6 system tray:

```
python

from PyQt6.QtWidgets import QSystemTrayIcon, QMenu
from PyQt6.QtGui import QIcon, QAction

tray = QSystemTrayIcon(QIcon("ui/assets/icon.png"))
menu = QMenu()
menu.addAction("Balance: 45 min")
menu.addAction("Open Dashboard")
menu.addAction("Exit")
tray.setContextMenu(menu)
tray.show()
```

Milestone test:

- App starts in system tray
- Right-click shows coin balance
- Click "Open Dashboard" shows full window with today's data

Phase 5 — Polish + Package (Week 5-6)

Goal: Works like a real app. Can be installed.

Features to add:

- Windows startup on boot (registry key)

- Toast notifications ("You earned 20 coins! 🎉")
- Streak tracker (study days in a row)
- Export to CSV (Power BI ready)
- PyInstaller → single .exe file

Build executable:

```
bash

pip install pyinstaller
pyinstaller --onefile --windowed --icon=ui/assets/icon.ico main.py
# Output: dist/ProductivityGuard.exe
```

Windows startup registry:

```
python

import winreg
key = winreg.OpenKey(winreg.HKEY_CURRENT_USER,
                    r"Software\Microsoft\Windows\CurrentVersion\Run",
                    0, winreg.KEY_SET_VALUE)
winreg.SetValueEx(key, "ProductivityGuard", 0, winreg.REG_SZ,
                 r"C:\path\to\ProductivityGuard.exe")
```

Database Schema

```
sql
```

```
CREATE TABLE sessions (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  start_time  DATETIME NOT NULL,  
  end_time    DATETIME,  
  app_name    TEXT,  
  window_title TEXT,  
  url         TEXT,  
  category    TEXT CHECK(category IN ('study','timepass','neutral','unknown')),  
  duration_seconds INTEGER  
);
```

```
CREATE TABLE coins (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  date        DATE NOT NULL UNIQUE,  
  earned_seconds INTEGER DEFAULT 0,  
  spent_seconds INTEGER DEFAULT 0,  
  balance_seconds INTEGER DEFAULT 0  
);
```

```
CREATE TABLE coin_transactions (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  timestamp   DATETIME DEFAULT CURRENT_TIMESTAMP,  
  type        TEXT CHECK(type IN ('earn','spend','expire')),  
  amount_seconds INTEGER,  
  source      TEXT,  
  note        TEXT  
);
```

```
CREATE TABLE study_list (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  identifier  TEXT NOT NULL,  
  type        TEXT CHECK(type IN ('app','domain'))  
);
```

```
CREATE TABLE block_list (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  identifier  TEXT NOT NULL,  
  type        TEXT CHECK(type IN ('app','domain')),  
  is_active  BOOLEAN DEFAULT 1  
);
```

Installation

```
bash

# 1. Clone repo
git clone https://github.com/yourusername/ProductivityGuard.git
cd ProductivityGuard

# 2. Create virtual environment
python -m venv venv
venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run (as Administrator for blocker to work)
python main.py
```

For browser extension:

1. Open Chrome → `chrome://extensions`
 2. Enable "Developer mode"
 3. Click "Load unpacked" → select the `extension/` folder
-

Requirements.txt

```
pywin32==306
pygetwindow==0.0.9
psutil==5.9.8
fastapi==0.110.0
uvicorn==0.27.1
PyQt6==6.6.1
schedule==1.2.1
pystray==0.19.5
Pillow==10.2.0
requests==2.31.0
python-dotenv==1.0.1
```

Configuration

Edit `config/settings.json`:

```
json
{
  "earn_rate": 6,
  "min_study_block_minutes": 15,
  "coin_expiry": "midnight",
  "max_daily_coins_minutes": 120,
  "min_spend_minutes": 5,
  "api_port": 5000,
  "poll_interval_seconds": 5
}
```

How This Was Built

This project was planned and built using:

- **Claude AI** (claude.ai) — architecture, planning, code generation
- **Cursor IDE** — AI-assisted coding environment
- **GitHub Copilot** — inline code completion (optional)

Recommended workflow for contributors:

1. Read `docs/PIPELINE.md` before starting any phase
 2. Work phase by phase — don't skip ahead
 3. Test each milestone before moving on
-

Roadmap

- ☒ Project architecture designed
- ☒ Database schema defined
- ☐ Phase 1: Activity tracker
- ☐ Phase 2: Coin engine
- ☐ Phase 3: App blocker
- ☐ Phase 4: Dashboard UI

- ☐ Phase 5: Packaging + polish
 - ☐ Future: ML-based smart classifier (LangChain)
 - ☐ Future: Mobile companion app
 - ☐ Future: Multi-device sync
-

Important Notes

- Run as **Administrator** for the hosts file blocker to work
 - The browser extension must be loaded manually (not from Chrome Web Store yet)
 - Coins reset at midnight by default — configure in `settings.json`
 - Windows only (Linux/Mac support not planned)
-

License

MIT License — free to use, modify, and distribute.

Built by a Data Science aspirant who got tired of wasting time and decided to build the solution instead.